University of Massachusetts, Amherst
CICS
COMPSCI 683: Artificial Intelligence

Assignment Bonus 1: Search, Games and CSPs

Important Instructions:

- Please upload your solution to Gradescope.
- You may work with a partner on your homework; however, if you choose to do so, **you must** write down the name of your partner on the assignment. In addition, you **may not submit similar/identical solutions**.
- Provide a mathematical proof/computation steps to all questions, unless explicitly told not to. Solutions with no proof/explanations (even correct ones!) will be awarded no points.
- Follow the guidelines on writing mathematical proofs provided on Canvas. Remember: we grade your proofs on correctness and clarity, not length. We will deduct points for incorrect claims, even if the proof as a whole is correct. For (a dramatic) example, if you conclude your proof with $1 + 1 = 4$, expect a grade deduction.
- You may look up combinatorial inequalities and mathematical formulas online, but please do not look up the solutions. Most problems can be found in research papers/online; if you happen upon the solution before figuring it out yourself, mention this in the submission. **Unreferenced copies of online material will be considered plagiarism.**

Question 1

1. (20 points) Bidirectional Breadth-First Search (BBFS) works as follows. We are given a search problem with a set of states $\mathcal{S}$, a set of actions A, a transition function $T : \mathcal{S} \times A \to \mathcal{S}$ and a cost function $c_s : A \to \mathbb{R}_{\geq 0}$ for each state $s \in \mathcal{S}$.

   We are given a starting state $s \in \mathcal{S}$ and a goal state $t \in \mathcal{S}$ and want to find a shortest path between them. We initialize two frontiers, i.e., lists of nodes (encoding the state plus some other stuff), called L_s and L_t. Initially, L_s only contains a node encoding the starting state s, and L_t contains a node encoding the goal state t.

   At every iteration i, we first check whether L_s and L_t have any nodes that represent states that have been expanded by the other list. If they do, then we terminate the search. Otherwise, we pick a node in L_s, expand it, and add its neighbors to the list L_s; then, we pick a node in L_t, expand it, and add its neighbors to the list L_t. We use a BFS mechanic to select nodes, i.e., the frontier lists use a queue data structure. There are two important points here.

   (i) When adding elements to the goal frontier L_t, we do not check whether a node $v \in L_t$ can reach each neighbors. Rather, the neighbors of nodes in $v \in L_t$ are the nodes that can reach v. When adding nodes to the frontier L_s, on the other hand, when adding the neighbors of $v \in L_s$, we add all nodes that can be reached from v by performing a single action. More formally, we define

$$N(v) = \{s \in \mathcal{S} : \exists a \in A(v.\texttt{state}), T(v.\texttt{state}, a) = s\}$$
$$N^{-1}(v) = \{s \in \mathcal{S} : \exists a \in A(s), T(s, a) = v.\texttt{state}\}.$$

   So, when selecting a node $v \in L_s$, we add $N(v)$ to L_s, whereas we add $N^{-1}(v)$ to L_t. We assume that we can compute both $N(v)$ and $N^{-1}(v)$ in constant time (in other words, ignore the time it takes to compute these sets in your analysis).

(ii) We do not perform goal checks; instead, we check whether the two frontiers intersect at any point and terminate once they do.

(a) (10 points) Assume that all transition costs are 1, i.e., that $c(s, a) = 1$. Prove that BBFS finds the shortest path from s to t.

(b) (10 points) Next, assume that transition costs are arbitrary. Bidirectional Uniform Cost Search works like BBFS, but instead of a queue, we implement a priority queue where the nodes with lowest path cost are expanded first. Prove that BUCS finds the shortest path from s to t.

Solution 1

Question 2

2. (20 points) We assume that our search space is over a finite set of states that are represented as vectors in $\mathbb{R}^n$. We assume that there is a unique goal state $\vec{t} \in \mathbb{R}^n$. Given two points $\vec{x}, \vec{y} \in \mathbb{R}^n$, let $|\vec{x} - \vec{y}|$ be the vector $(|x_1 - y_1|, \ldots, |x_n - y_n|)$. Given a vector $\vec{q} \in \mathbb{R}^n_+$, we define the *harmonic function* as follows:

$$H(\vec{q}) = \begin{cases} \frac{1}{\sum_{i=1}^{n} \frac{1}{q_i}} & \text{if } q_i > 0 \text{ for all } i \\ 0 & \text{otherwise.} \end{cases}$$

   (a) (10 points) We define a heuristic over the search space as follows: given a state $\vec{x} \in \mathbb{R}^n$, let $h_1(\vec{x}) = H(|\vec{x} - \vec{t}|)$. Determine whether this heuristic is consistent, admissible or neither.

   (b) (10 points) Consider the heuristic $h_2(\vec{x}) = \min_i |x_i - t_i|$. Which of the following is true: h_1 dominates h_2, h_2 dominates h_1, or neither? Explain your answer.

Solution 2

Question 3

3. (20 points) The k-coloring problem is defined as follows. We are given an undirected graph G with a vertex set V of n vertices, and an edge set E. We can assign one of k colors to each one of the nodes, with the requirement that two adjacent vertices cannot be the same color.
   (a) (20 points) Describe a dynamic programming algorithm that computes a valid k coloring for a given graph G. The algorithm can run in time exponential in n. **Hint:** given a subset of vertices $S \subseteq N$, let $C(S, i)$ be a boolean variable which corresponds to "the subgraph induced by the set S can be colored with i colors". Can you derive a formula for $C(S, i)$ as a function of $C(T, j)$ for all $T \subset S$?
   (b) (bonus points) Prove that the k-coloring problem is NP complete when $k \geq 3$.

Solution 3

Question 4

4. (30 points) Let us consider a simplified version of poker. We have two players and four cards with the numbers $1, 2, 3, 4$ on them. The four cards are shuffled uniformly, i.e., we select an ordering of the cards uniformly at random. Player 1 draws a card, and then player 2 draws a card (in particular, players can't draw the same card). Players can see their own card, but not the other player's card. Players can choose to either call or fold, and do so one at a time: player 1 speaks first, and then player 2. Gameplay works as follows:

   (I) If player 1 folds, then the game ends. Player 2 gets 1 point and player 1 gets 0 points.

   (II) If player 1 calls, then it's player 2's turn.

      (i) If player 2 calls as well, then the player whose card has a higher value gets 2 points and the player whose card has a lower value gets -1 points.

      (ii) If player 2 folds, then player 1 gets 1 point and player 2 gets 0 points.

   (a) (10 points) Draw the game subtree for this game, when player 1 receives a card whose value is 2 and player 2 gets a card whose value is 3. You should indicate both players' information sets; for example, you should include nodes that signify that the player 1 did not receive a card whose value is 2. However, you don't have to write down their subtrees (this would be extremely tedious!).

   (b) (10 points) Write down the utility function of Player 2 from calling/folding, for every possible card type they get.

   (c) (10 points) What is the equilibrium strategy for player 1, given that the card they got has a value of 1?

   (d) (10 points) Identify one pure strategy Bayes-Nash equilibrium of this game if one exists, or explain why no pure strategy Bayes-Nash equilibrium exists.

Solution 4